420-511-VA Web Services

Week 2: The Foundations of the Web and HTTP

Pravish Sainath

Course Map

Week 1 Recap

- Processes communicate through sockets.
- TCP gives an ordered byte stream.
- Applications still need framing, encoding, errors, and timeouts.

Week 2 Idea

- HTTP is a standardized application protocol.
- The Web adds URIs, methods, status codes, fields, representations, caching, and security expectations.

TCP socket → **HTTP message** → **Web service**

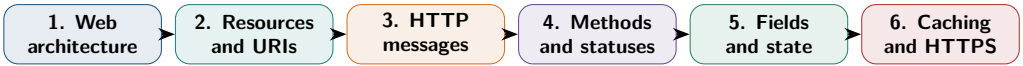
Takeaway: This week turns raw client/server communication into the conventions used by browsers, servers, APIs, proxies, and caches.

Learning Outcomes

By the end of the week, students should be able to:

1. distinguish the Internet, the Web, a Web application, and a Web API;
2. parse a URI and explain the roles of scheme, authority, path, query, and fragment;
3. explain URI, URL, and URN without treating them as identical;
4. read HTTP/1.1 request and response messages;
5. choose HTTP methods using safety and idempotence;
6. interpret common status codes and status-code classes;
7. identify the purpose of core HTTP fields;
8. explain how HTTP can be stateless while applications still have sessions;
9. reason about cookies, caching, validation, and HTTPS.

Roadmap for the Class



Teaching stance

We will use everyday systems – school portals, Instagram-like feeds, ticketing, shopping carts, and weather pages – to connect each technical idea to something students already understand.

1. Web Architecture

From sockets and hosts to user agents, origins, and intermediaries

Internet vs. Web vs. Web Application

Internet

Global network infrastructure: IP addresses, routing, transport protocols, and connected hosts.

Web

An application system on top of the Internet, built around URLs/URIs, HTTP, links, documents, resources, and representations.

Web app / API

A service that users or programs interact with through Web conventions. Examples: Omnivox-like portal, GitHub API, weather service.

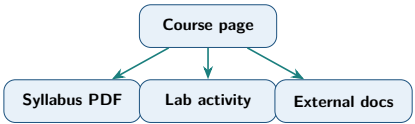
Takeaway: The Web is not the whole Internet. It is a very successful application layer built on Internet communication.

The Web's Original Idea: Hypertext at Internet Scale

- Pages and APIs can point to other resources.
- A link uses an identifier: usually a URL.
- A client retrieves a representation through a protocol.
- Servers can be independently owned and distributed.

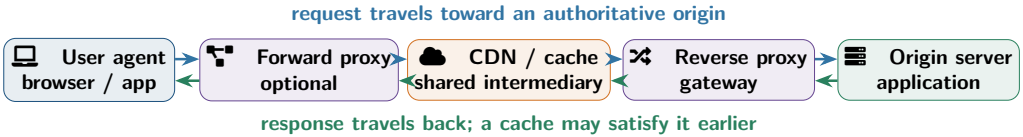
Everyday analogy

A course page links to a syllabus PDF, Moodle activity, assignment upload, and official Java docs. The student sees one connected experience; multiple servers may be involved.



The Web is a graph of linked resources, not one central machine.

Core Components in a Web Interaction



Takeaway: HTTP is designed for message forwarding through intermediaries, not just direct browser-to-server conversations.

Why Intermediaries Exist

Intermediary	What it does	Everyday example
Forward proxy	Acts for clients.	School/corporate outbound filter.
CDN/shared cache	Serves reusable responses near users.	Course images, CSS, JavaScript.
Reverse proxy	Stands in front of origins.	Routes `/api` to an API and `/static` to assets.
Gateway/API gateway	Secures, translates, aggregates, or controls access.	Portal gateway to registration, grades, payments.

Important consequence

A failure may be in browser, network, cache, gateway, origin app, database, or upstream service. HTTP status codes help narrow where the observed failure appears.

What Happens When You Enter an HTTPS URL?

1. Browser parses the URL and identifies scheme, host, port, path, query, and fragment.
2. Browser resolves the host name to one or more IP addresses.
3. Browser opens a connection to the server endpoint, often through several network hops.
4. TLS establishes an encrypted and authenticated channel for HTTPS.
5. Browser sends an HTTP request.
6. Server or cache returns an HTTP response.
7. Browser interprets headers and representation content, then may request linked resources.

Course portal example

Typing `https://portal.example.edu/courses/420-511?term=F26` can cause requests for HTML, CSS, JavaScript, icons, course data, login/session checks, and cached assets.

2. Resources, Representations, and URIs

How the Web names things and retrieves useful views of them

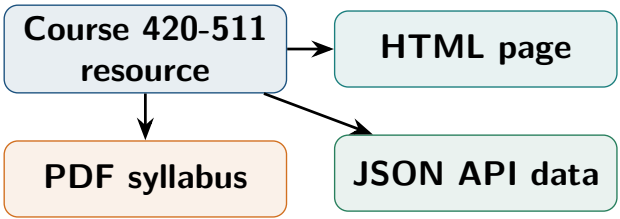
Resources and Representations

Resource

Anything identifiable on the Web: a course, profile, image, ticket, weather forecast, API collection, or document.

Representation

A transferable form of resource state: HTML, JSON, PDF, image, XML, or cached response.



Takeaway: Resource = conceptual target. Representation = form returned in the response.

URI Anatomy: Parse the Identifier Before You Reason



- The authority names who controls the resource name; the path names a target under that authority.
- Query data begins with ?; the fragment begins with # and is normally handled by the client after retrieval.

URI Components in Practice

Component	Role	Course portal example
Scheme	What protocol or naming scheme is used.	`https` means use HTTP over TLS.
Authority	Who has authority over the resource name.	`portal.example.edu:443`.
Path	Hierarchical resource location within that authority.	`/courses/420-511`.
Query	Extra parameters for the request.	`term=F26` selects the semester view.
Fragment	Client-side secondary target.	`schedule` jumps to the schedule section.

Takeaway: Do not parse a URI by guessing from English words. Use the separators and component rules.

URI vs. URL vs. URN

URI

Uniform Resource Identifier. General category: identifies a resource.

Question: "What resource are we talking about?"

URL

Uniform Resource Locator. A URI that gives a retrieval location.

`https://portal.example.edu/courses/420-511`

URN

Uniform Resource Name. A persistent namespace name, not one current server path.

`urn:isbn:9780131103627`

Everyday examples

- **School portal URL:** a course page retrievable now.
- **ISBN/URN:** a book name independent of one library or store.
- **Fragment reference:** `#schedule` names a section in the current representation.

Key rule: every URL is a URI; not every URI is a URL.

Everyday Examples: URI, URL, URN

URL examples: locator + identifier

`https://portal.example.edu/courses/420-511`

Course page at a retrievable Web location.

`https://shop.example/items/45?color=blue`

Product view selected with a query.

URN and URI-reference examples

`urn:isbn:9780131103627`

Book identified by ISBN, independent of one store.

`#schedule`

Fragment reference inside the current document.

Memory aid

Every URL is a URI. A URN is a URI that names within a persistent namespace. A fragment is a secondary target in a representation.

Origin: Scheme + Host + Port

Origin

For Web security and browser policy, an origin is the tuple:

$$(\text{scheme}, \text{host}, \text{port})$$

Path and query do not define the origin.

Compared with base	Same origin?
`https://portal.../courses`	base
`https://portal.../grades`	yes: path differs only
`http://portal.../courses`	no: scheme differs
`https://api.../courses`	no: host differs
`https://portal...:8443`	no: port differs

Takeaway: Same origin is central to browser security, cookies, and cross-origin requests.

Practical URI Rules for Students

Good habits

- Choose meaningful resource paths.
- Keep identifiers stable when possible.
- Use query parameters for filtering, searching, paging, or optional selection.
- Percent-encode special characters when they are data, not separators.

Avoid

- Spaces and ambiguous punctuation.
- Secrets in URLs.
- Changing URLs whenever display text changes.
- Assuming a fragment is sent to the server like a query parameter.

Example

Better: `/courses/420-511/assignments?term=F26` Less clear: `/getStuff?id=420511&x=fall`

3. HTTP Messages

What travels between the user agent, intermediaries, and origin

HTTP as the Application Protocol of the Web

HTTP defines

- request and response structure;
- methods expressing client intent;
- status codes expressing outcomes;
- fields/headers carrying metadata;
- representation metadata and caching controls.

HTTP does not define

- your database schema;
- your application business rules;
- exact UI behavior;
- whether a payment is safe to retry;
- all authorization policies.

Takeaway: HTTP gives common semantics, but application correctness still belongs to the application.

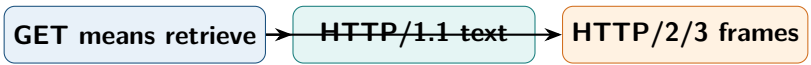
HTTP Semantics vs. Wire Format

Semantics

The meaning of methods, status codes, fields, caching, conditional requests, and representation metadata. These ideas are mostly shared across HTTP versions.

Wire format

How bytes travel on the connection. HTTP/1.1 uses textual start lines and header fields; HTTP/2 and HTTP/3 use binary framing.



HTTP Versions: Same Semantics, Different Transport

	HTTP/1.1	HTTP/2	HTTP/3
Format	Textual start line and fields.	Binary framing over TCP.	Binary framing over QUIC/UDP.
Concurrency	Several requests usually need several connections or careful pipelining.	Multiplexed streams on one connection.	Multiplexed streams with QUIC behavior.
Student focus	Easiest for learning message anatomy.	Important modern deployment context.	Common for large Web services.

Course focus

We learn HTTP using HTTP/1.1 notation because it is readable. The method/status/header semantics still matter in HTTP/2 and HTTP/3.

HTTP/1.1 Request Anatomy


```
GET /courses/420-511?term=F26 HTTP/1.1
Host: portal.example.edu
Accept: text/html
User-Agent: ExampleBrowser/1.0
Cookie: session=abc123
```

- **Request line**: method, target, version.
- **Fields**: metadata and control information.
- **Blank line**: separates fields from content.
- **Content**: optional body for methods such as POST or PUT.

Takeaway: A request asks for an operation on a target resource, with fields that explain context and preferences.

HTTP/1.1 Response Anatomy

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: private, max-age=60
ETag: "course-v17"

<html>...</html>
```

- **Status line**: version, status code, reason phrase.
- **Fields**: content type, caching, cookies, redirection, authentication, etc.
- **Content**: representation of the resource or problem details.

Takeaway: The status code tells the client how to interpret the result before it looks at the body.

A POST Request With Content


```
POST /registrations HTTP/1.1
Host: portal.example.edu
Content-Type: application/json
Accept: application/json
Cookie: session=abc123

{"course":"420-511","term":"F26"}
```

What to notice

- The method expresses intent: create or process a registration request.
- `Content-Type` describes the request body format.
- `Accept` describes what response format the client prefers.
- The cookie may carry a session identifier.

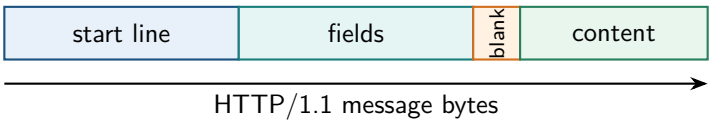
HTTP/1.1 Framing Essentials

Message fields end

Headers end at the empty line. Everything after that belongs to message content if content is expected.

Content length

`Content-Length` or transfer coding tells the receiver where content ends in HTTP/1.1.



Takeaway: This is the HTTP version of the Week 1 framing problem.

4. Methods and Status Codes

How clients express intent and servers report outcomes

HTTP Methods Express Client Intent

Method

An HTTP method tells the server the kind of action the client wants to perform on the target resource. It is not merely part of the URL string.

Everyday portal examples

- `GET /courses/420-511`: view course data.
- `POST /registrations`: submit a registration request.
- `DELETE /cart/items/7`: remove a selected cart item.

Why it matters

- Caches rely on method semantics.
- Browsers and tools handle methods differently.
- APIs become easier to document and test.

Safe and Idempotent: Two Crucial Properties

Property	Meaning	Example
Safe	The client is only asking for retrieval; it should not request a state-changing action.	Refreshing a course page with `GET`.
Idempotent	Repeating the same request should have the same intended effect as making it once.	Sending `PUT /profile` with the same full profile twice.
Neither safe nor idempotent	Repetition may create multiple effects.	Clicking `POST /orders` twice may create two orders unless the API prevents it.

Exam trap

Idempotent does not mean "same response every time". It means same intended server-side effect. Logs and timestamps can still differ.

Core Methods to Reason About First

Method	Safe?	Idempotent?	Typical use
GET	yes	yes	Retrieve a representation.
HEAD	yes	yes	Retrieve metadata without response body.
POST	no	no by default	Submit data to be processed; create subordinate resources; actions.
PUT	no	yes	Replace or create the target resource with supplied representation.
PATCH	no	not guaranteed	Partially modify resource state.
DELETE	no	yes	Remove the target resource when authorized.
OPTIONS	yes	yes	Ask about communication options or capabilities.

Takeaway: Choose a method based on request meaning, not based on what is easiest to code.

Method Selection Examples

User action	Likely method	Why
Open course schedule.	GET	Retrieves a representation; safe and cacheable in principle.
Submit registration request.	POST	Creates/processes a new action under a collection.
Replace complete profile.	PUT	Client sends the full new representation of the profile.
Change only notification preference.	PATCH	Partial update to existing profile.
Remove a saved item from cart.	DELETE	Delete the target cart item.
Check allowed methods before using an endpoint.	OPTIONS	Capability inquiry.

Status-Code Classes

1xx Informational
request is still in progress

2xx Success
request accepted or completed

3xx Redirection
use cache or another target

4xx Client/request issue
target, syntax, auth, validation

5xx Server/gateway issue
origin, gateway, upstream

How to read a status code

First identify the class, then the specific code. A `404` is not just "something broke"; it says the target resource was not found or is not being revealed. A `500` says the request reached a server-side failure boundary.

Core Status Codes I: Success, Redirects, Caching

Code	Meaning in practical terms	Everyday example
`200 OK`	Request succeeded and a response representation is included.	Course page, JSON profile, or image returned.
`201 Created`	A new resource was created; `Location` often identifies it.	Registration record or uploaded assignment created.
`204 No Content`	Request succeeded, but there is intentionally no body.	Delete saved item or update preference.
`301/308`	Permanent redirect to a canonical or moved URI.	Old course URL moved to new portal path.
`302/303/307`	Temporary redirect variants.	Login flow redirects after authentication.
`304 Not Modified`	Cached representation is still valid; reuse local copy.	Browser reuses cached logo or CSS.

Core Status Codes II: Request and Server Problems

Code	Meaning in practical terms	Everyday example
`400 Bad Request`	Request cannot be parsed or is structurally wrong.	Broken JSON or invalid request syntax.
`401 Unauthorized`	Authentication is missing, invalid, or required.	Missing login token.
`403 Forbidden`	Caller is known but not allowed to perform the action.	Student tries admin-only page.
`404 Not Found`	Resource is not found or not disclosed to this caller.	Unknown course ID.
`409 Conflict`	Request conflicts with current resource state.	Seat already reserved by another buyer.
`422 Unprocessable Content`	Syntax understood, but semantic validation failed.	Registration field missing or course section invalid.
`429 Too Many Requests`	Rate limit or quota exceeded.	Too many login attempts.
`500/502/503/504`	Origin/gateway/unavailable/upstream-timeout failures.	Application bug, down service, or slow upstream.

Three Status Distinctions That Matter

400 vs. 422

`400`: request cannot be parsed or is structurally wrong.
`422`: structure is understood but semantic validation fails.

401 vs. 403

`401`: authenticate first or credentials invalid.
`403`: identity may be known, but action is not allowed.

502 vs. 504

`502`: gateway received a bad upstream response.
`504`: gateway timed out waiting upstream.

Takeaway: Good status codes make debugging and API clients much more predictable.

5. HTTP Fields, State, and Cookies

Metadata, statelessness, and how applications remember users

HTTP Fields: Metadata and Control Information

HTTP field / header

An HTTP field is a named piece of metadata in a request or response. Fields describe context, preferences, content, caching, authentication, cookies, connection behavior, and more.

Request fields

- `Host`: intended authority.
- `Accept`: preferred response media type.
- `Authorization`: credentials or token.
- `Cookie`: state previously set by server.

Response fields

- `Content-Type`: representation format.
- `Cache-Control`: caching policy.
- `Set-Cookie`: store a cookie.
- `Location`: redirect or created resource URI.

Essential HTTP Fields by Purpose

Purpose	Fields	What students should ask
Routing/context	`Host`, `Forwarded`	Which origin or proxy context is involved?
Representation metadata	`Content-Type`, `Content-Length`	What format and size is the content?
Negotiation	`Accept`, `Accept-Language`	What response form does the client prefer?
Caching	`Cache-Control`, `ETag`, `Last-Modified`	Can a response be reused? How is it validated?
Conditional requests	`If-None-Match`, `If-Modified-Since`	Is cached content still current?
Cookies/sessions	`Cookie`, `Set-Cookie`	What state is carried by the browser?
Security/auth	`Authorization`, `WWW-Authenticate`	How does the server challenge or receive credentials?
Redirection/retry	`Location`, `Retry-After`	Where should the client go next or when try again?

Three Header Rules to Keep Straight

1. **Fields describe messages; they do not replace application validation.**

   A `Content-Type: application/json` field does not prove the body is valid JSON.

2. **Request fields and response fields have different roles.**

   `Accept` appears in a request; `Content-Type` usually describes actual content in request or response.

3. **Some fields influence intermediaries.**

   Caching and connection-related fields can change behavior before the message reaches the origin application.

Example

A browser may send `If-None-Match: "course-v17"`. A server can reply `304 Not Modified`, so the browser reuses its cached representation.

Four Kinds of State

Kind	Where it lives	Example
Resource state	Authoritative server/database.	Course seats, grades, profile record.
Representation state	Response document sent to client/cache.	HTML or JSON snapshot of course schedule.
Application session state	Server-side or distributed session store, associated with user/session ID.	Logged-in portal session.
Client/UI state	Browser/app memory or storage.	Open tab, dark mode, selected filter.

Takeaway: "State" is not one thing. Clarify which state you mean before reasoning about HTTP statelessness.

HTTP Statelessness

Stateless communication

Each HTTP request should contain enough information for the server to understand and process it in context. The protocol does not require the server to remember a transport conversation between requests.

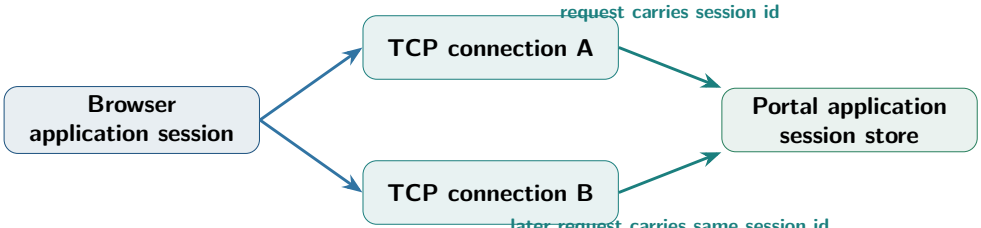
Does not mean

- the server has no database;
- applications cannot have login sessions;
- users cannot have carts;
- every connection carries only one request.

It means

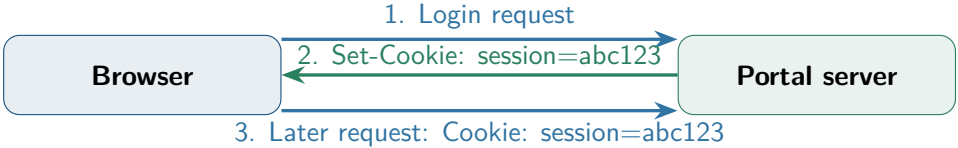
- request meaning is not hidden in the transport connection;
- a later request must carry the needed context;
- scaling and intermediaries become easier.

TCP Connection Is Not the Same as User Session



Example: a school-portal login can survive a Wi-Fi reconnection, a new browser tab, or a new TCP connection because the application session is identified at the HTTP/application layer.

Cookies: A Browser-Carried State Mechanism



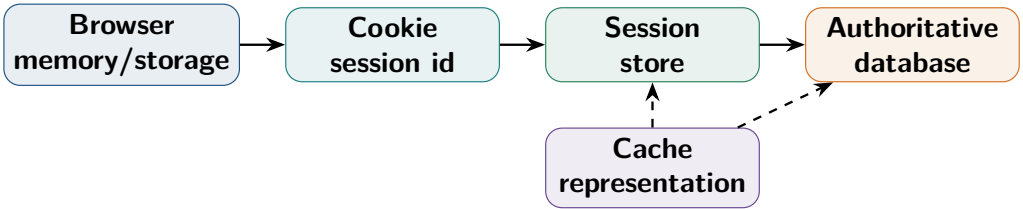
- The cookie is stored by the browser and sent back on matching requests.
- The cookie value should usually be an opaque identifier, not sensitive personal data.
- Attributes such as `Secure`, `HttpOnly`, and `SameSite` change browser behavior.

Everyday Cookie Examples

Application	Cookie can carry or select	Why it matters
School portal	Session identifier, preferred language, selected term.	The portal recognizes later clicks without asking for login again.
Shopping site	Cart/session identifier.	Items remain in the cart while the user browses.
Streaming/news site	Signed-in session, region, theme, consent, paywall meter.	The page remains personalized and consistent.
Experimentation	A/B test bucket.	The same user keeps seeing the same experiment version.

Takeaway: A cookie is not the whole session. It is usually a small browser-carried clue that lets the server find or apply application state.

Where Application State Can Live



Ticketing example

A browser may store a session cookie; the server session remembers login and checkout progress; the database stores authoritative seat ownership; a cache may store only availability pages.

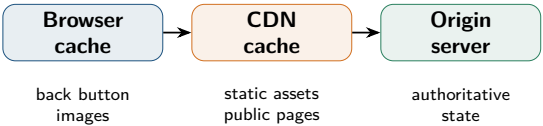
6. Caching and HTTPS

Performance, validation, and secure transport

Why HTTP Caching Matters

Caching

Reusing a stored response for a later request when HTTP rules allow it.



- lowers latency;
- reduces origin load;
- saves bandwidth;
- improves user experience.

Caching is not a lock

A cached “2 seats available” page can be stale. The reservation must check authoritative state.

Fresh, Stale, or Unusable?



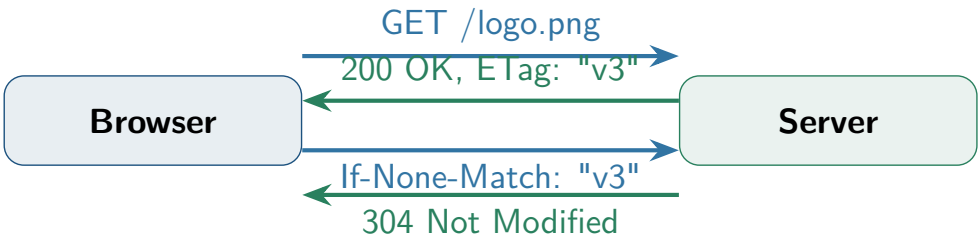
- Freshness is often controlled with `Cache-Control: max-age=...`.
- Validation uses fields such as `ETag` and `Last-Modified`.
- A successful validation can produce `304 Not Modified`.

Cache-Control Essentials

Directive	Meaning	Use case
`max-age=60`	Response is fresh for 60 seconds.	Course listing that changes occasionally.
`public`	Shared caches may store it.	Public CSS, images, documentation.
`private`	Browser may store; shared caches should not.	Personalized dashboard HTML.
`no-cache`	Store allowed, but must revalidate before reuse.	Data that should be checked each time.
`no-store`	Do not store at all.	Sensitive banking or private records.

Takeaway: "no-cache" does not mean "do not store". It means "do not reuse without validation".

ETag and Last-Modified: Avoid Downloading Again



Takeaway: Validation can save bandwidth while still checking whether cached content is current.

HTTPS: HTTP Over a Secure Transport

TLS provides

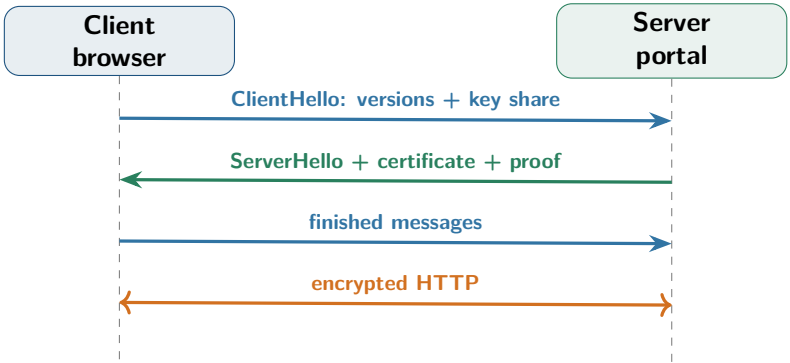
- confidentiality: outsiders cannot read content;
- integrity: tampering is detected;
- server authentication: browser can verify the site identity through certificates.

Still required

- application authentication;
- authorization checks;
- input validation;
- secure cookie settings;
- safe server-side code.

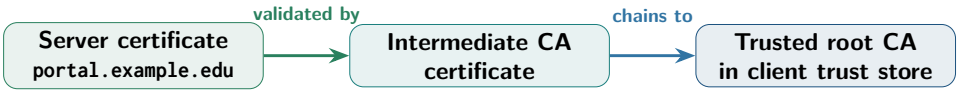
Takeaway: HTTPS protects the connection. It does not automatically make an application correct or authorized.

Simplified TLS 1.3 Handshake



Takeaway: The browser establishes keys and verifies the server before sending protected HTTP application data.

Certificate Chain: Why the Browser Trusts the Server



- The certificate must match the host name the user requested.
- The chain must lead to a trusted root and must not be expired or revoked.
- Certificate warnings indicate a trust problem, not just a visual annoyance.

What HTTPS Does Not Guarantee

HTTPS does help with

- protecting credentials in transit;
- preventing network observers from reading page content;
- detecting tampering on the connection;
- verifying the server identity, if certificate validation succeeds.

HTTPS does not prove

- the site is honest;
- the code is bug-free;
- the user is authorized;
- cached content is current;
- a form submission is safe to retry.

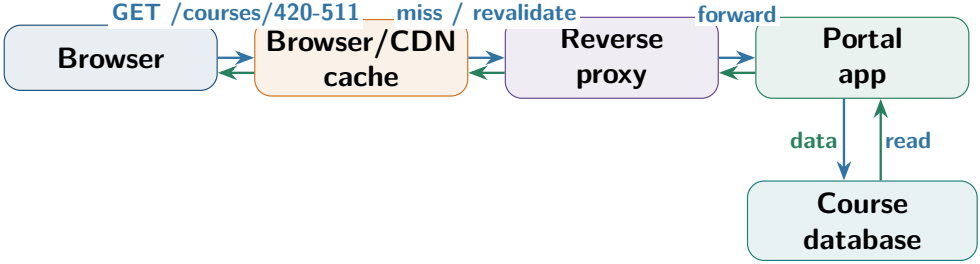
Security mindset

HTTPS is necessary for modern Web applications, but it is only one layer of protection.

7. End-to-End Reasoning

Apply the concepts to everyday Web systems

Scenario 1: Opening a College Course Page



URI identifies the course page. `GET` asks for a representation. Cache fields decide reuse. `200`, `304`, and `404` explain different outcomes.

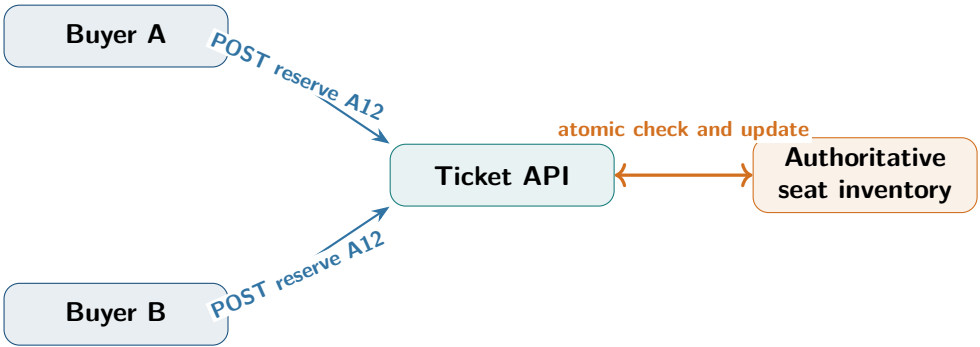
Scenario 2: Registering for a Course

1. Student logs in; portal sets a session cookie.
2. Browser sends `POST /registrations` with JSON content.
3. Server validates authentication, authorization, deadlines, prerequisites, and seats.
4. Server returns `201`, `409`, `422`, `401`, or `403` depending on outcome.

Reasoning examples

- Missing login: `401`.
- Student cannot take the course: `403` or domain-specific rejection.
- Seat taken during submission: `409`.
- Bad JSON: `400`.
- Valid JSON but missing required field: `422`.

Scenario 3: Ticketing Under Concurrency



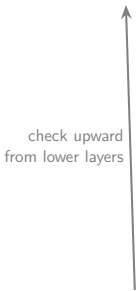
A cached availability page is informational. Only the authoritative mutation decides who gets the scarce seat.

Takeaway: Caching improves reads; concurrency control protects state-changing operations.

Common Misconceptions

Misconception	Correction
"HTTPS means the application is secure."	HTTPS protects the connection; the application still needs validation, authentication, authorization, and safe code.
"HTTP is stateful because I stay logged in."	HTTP itself is stateless; the application carries context with cookies, tokens, and server-side state.
"GET is only for HTML pages."	GET retrieves representations: HTML, JSON, images, PDFs, CSS, JavaScript, and more.
"404 means the server is down."	404 is about the target resource. Server outages are usually 5xx.
"no-cache means do not store."	It means store allowed, but revalidate before reuse. Use `no-store` to prohibit storage.

Systematic HTTP Diagnostic Reasoning



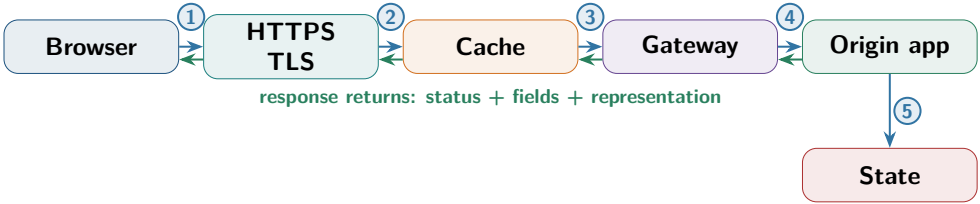
6. **Application meaning: validation, authorization, state, business rules**
5. **HTTP: method, URI, status, fields, content, cache**
4. **HTTPS/TLS: certificate, encrypted channel, trust chain**
3. **Transport: connection, timeout, reset, multiplexing**
2. **Name/address: host resolution, origin, port**
1. **Process/service: browser, proxy, origin, database actually running**

Takeaway: Avoid random debugging. Identify which layer first shows evidence of failure.

8. Consolidation

The complete mental model for Week 2

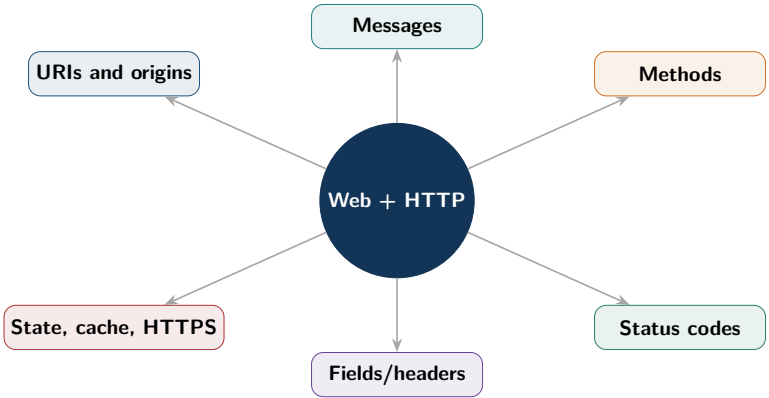
One Integrated HTTP Exchange



1 URI, method, fields | **2** encrypted HTTP | **3** cache hit/miss/validation
4 gateway forwards | **5** origin reads or updates authoritative state

- Methods express intent; status codes express outcomes; fields guide interpretation.
- Caches reuse responses under rules; HTTPS protects transport; application state remains authoritative elsewhere.

Week 2 Concept Map



Complete mental model:

HTTP gives shared semantics for identifiers, methods, status codes, fields, caching, and secure transport.

Key Points to Remember I

- The Web is an application layer on top of Internet communication.
- A Web interaction can involve a browser, DNS, TLS, caches, proxies, gateways, origin applications, and databases.
- A URI identifies a resource; a URL locates it; a URN names it in a persistent namespace.
- A resource is not the same as its representation.
- HTTP messages have method/status information, fields, and optional content.
- HTTP semantics are the main teaching focus; HTTP/1.1 notation is readable, but HTTP/2 and HTTP/3 preserve much of the same meaning.

Key Points to Remember II

- Methods should be chosen according to intent, safety, and idempotence.
- Status codes should describe the outcome accurately enough for clients and developers to respond.
- Headers carry metadata and control information; they do not replace validation.
- HTTP is stateless, but applications can maintain sessions using cookies, tokens, and server-side stores.
- Caching improves performance but must respect freshness, validation, privacy, and authoritative state.
- HTTPS protects the connection but does not automatically secure the application logic.

Class Review Questions

1. What is the difference between the Internet and the Web?
2. In the example URI, which part is the authority and which part is the path?
3. Why is a resource not the same as a representation?
4. When should an API use `POST` rather than `GET`?
5. Why is `PUT` idempotent but `POST` not necessarily idempotent?
6. When would `401`, `403`, `404`, `409`, and `422` be used?
7. How can HTTP be stateless while a user stays logged in?
8. Why can `no-cache` still allow storage?
9. What does HTTPS protect, and what does it not protect?

References and Useful Reads

- RFC 3986 – Uniform Resource Identifier generic syntax: rfc-editor.org/rfc/rfc3986.html
- RFC 9110 – HTTP Semantics: rfc-editor.org/rfc/rfc9110.html
- RFC 9111 – HTTP Caching: rfc-editor.org/rfc/rfc9111.html
- RFC 9112 – HTTP/1.1: rfc-editor.org/rfc/rfc9112.html
- RFC 9113 – HTTP/2: rfc-editor.org/rfc/rfc9113.html
- RFC 9114 – HTTP/3: rfc-editor.org/rfc/rfc9114.html
- RFC 8446 – TLS 1.3: rfc-editor.org/rfc/rfc8446.html
- IANA HTTP status codes and methods: status registry, method registry
- MDN HTTP overview and guides: developer.mozilla.org/en-US/docs/Web/HTTP
- Kurose and Ross networking teaching resources: gaia.cs.umass.edu/kurose_ross/ppt.php